

1. Initial Request & Solution Anchor

TURN 1



Developer

Initial Request

"how to implement filter by date option (default is this month's expenses) in ruby on rails?"

[Attached: _expenses_table.html.erb]



ChatGPT

Solution Anchor

```
# [Date calculation omitted]
def index
  @expenses = Expense.where(datetime:
    start_date..end_date)
end
```

Defect: Unscoped Query

Directly queries all Expense records globally without user scoping, auth guard, or ordering.

2. Later-Stated Constraints (LSRI)

TURN 2



Developer

Follow-up Request

"here is expenses_controller"

```
def index
  @expenses = user_signed_in? ?
    current_user.expenses.order(...) : Expense.none
end
```

U1 User-Specific Scope

Actor & Perm

EEO=Yes

Revised

Restrict query to `current_user.expenses` (absent in Turn 1).

U2 Signed-Out Behavior

Actor & Perm

EEO=Yes

Revised

Return empty relation `Expense.none` when not signed in.

U3 Descending Ordering

Functional

EEO=Yes

Extended

Preserve sort: `order(datetime: :desc, created_at: :desc)`.

Earlier Elicitation Opportunity

"Whose expenses should be listed (current user only?), and how to handle unauthenticated users?"

3. Solution Revision

TURN 2



ChatGPT

Solution Revised

```
# [Date calculation preserved]
def index
  if user_signed_in?
    @expenses =
      current_user.expenses.where(...).order(...)
  else
    @expenses = Expense.none
  end
end
```

U1 Scope: Replaced global Expense with `current_user.expenses`.

U2 Auth Guard: Added `user_signed_in?` check with `Expense.none` fallback.

U3 Ordering: Preserved multi-column sort: `datetime: :desc, created_at: :desc`.

Takeaway

Proactively eliciting tacit constraints before code generation eliminates premature anchors and subsequent rework.